StoreOS — Full Build Specification

Universal Offline-First Store Management System

This document is the complete build brief for Antigravity. It combines the original product brief with the concrete architecture, schemas, phasing, and risk plan needed to actually start building — not just a prompt, but a spec an agent can execute against.

---

1. Product Summary

StoreOS is a store management / POS system that works for clothing, grocery, electronics, cosmetics, and furniture retailers alike, via a configurable product-attribute system rather than a vertical-specific data model.

Non-negotiable constraint: the app is **offline-first**. After initial setup/login, POS, inventory, customers, payments, due collection, and receipt printing must all work with zero internet connection, then sync when connectivity returns.

Platforms: responsive web app, installable PWA, works on desktop browsers, Windows, and mobile browsers from one codebase.

---

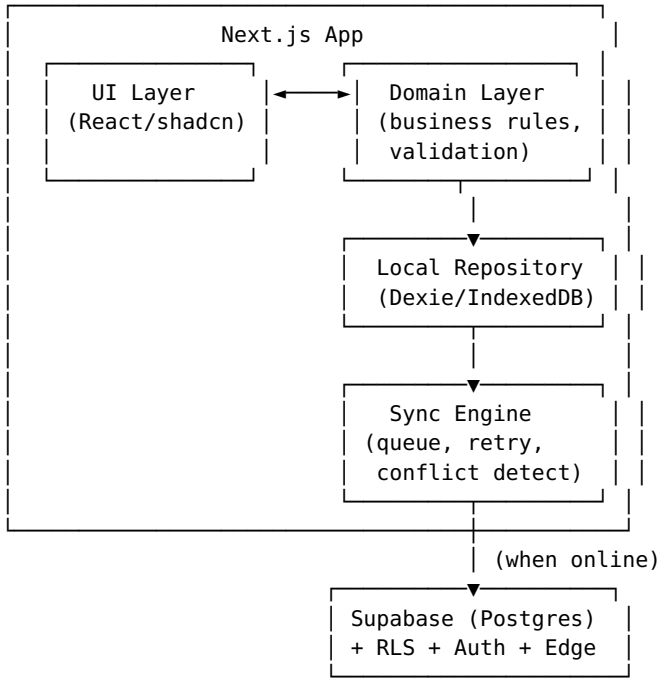
2. Tech Stack (confirmed)

Layer	Choice	Why
Framework	Next.js (App Router), TypeScript strict	SSR for first load, client-heavy runtime for offline logic
UI	Tailwind CSS + shadcn/ui	Fast, consistent, accessible components
Local DB	Dexie.js over IndexedDB	Structured, queryable offline storage with migrations
Cloud DB	Supabase (PostgreSQL)	Managed Postgres + Row Level Security + auth
Charts	Recharts	Dashboard/report visualizations
Offline shell	Service Worker (Workbox or hand-rolled)	App-shell caching + PWA installability
Auth	Supabase Auth (cached session for offline use)	Avoids rolling custom auth

---

3. System Architecture

Pattern: local-first with a sync queue, not "online app with a cache."



Key rule: every write in the app goes through the Local Repository first and returns immediately. The Sync Engine is a background process — the UI never blocks on network.

---

4. Folder Structure

```
storeos/
├── app/                          # Next.js App Router
│   ├── (auth)/login, /setup
│   ├── (dashboard)/dashboard
│   ├── (pos)/pos
│   ├── (inventory)/products, /categories, /stock
│   ├── (customers)/customers, /due
│   ├── (suppliers)/suppliers, /purchases
│   ├── (sales)/invoices, /returns
│   ├── (finance)/expenses
│   ├── (hr)/employees
│   ├── (reports)/reports
│   ├── (branches)/branches
│   └── (settings)/settings, /permissions, /backup
├── components/
│   ├── ui/                       # shadcn primitives
│   ├── pos/
│   ├── inventory/
│   ├── reports/
│   └── shared/
├── lib/
│   ├── db/
│   │   ├── dexie/                # local schema + migrations
│   │   └── supabase/             # client, server, RLS helpers
│   ├── sync/
│   │   ├── queue.ts
│   │   ├── resolver.ts           # conflict resolution
│   │   └── worker.ts
│   ├── domain/
│   │   ├── pos/
```

```
│   │   ├── inventory/
│   │   ├── payments/
│   │   └── money.ts            # integer minor-unit math
│   ├── validation/              # zod schemas, shared client+server
│   └── auth/
├── public/
│   ├── sw.js                   # service worker
│   └── manifest.json
├── tests/
│   ├── unit/
│   ├── integration/
│   └── sync/
└── docs/
    ├── ARCHITECTURE.md
    ├── SCHEMA.md
    └── LIMITATIONS.md
```

---

5. Cloud Schema (Supabase / Postgres — core tables)

All monetary columns are `bigint` storing **minor units** (e.g. poisha/cents). All tables carry `business_id` for multi-tenant isolation and `branch_id` where branch-scoped.

```
businesses(id, name, owner_id, created_at)
branches(id, business_id, name, address, is_active)
users(id, business_id, role, branch_id, full_name)        -- maps to
Supabase auth.users
roles_permissions(role, permission_key)

categories(id, business_id, name, parent_id)
products(id, business_id, sku, barcode, name, category_id, brand,
         unit, purchase_price_minor, sell_price_minor, tax_rate,
         has_variants, has_batch_tracking, has_serial_tracking,
is_active)
product_variants(id, product_id, attribute_json, sku, barcode,
extra_price_minor)
batches(id, product_id/variant_id, batch_no, expiry_date, qty)
serials(id, product_id/variant_id, serial_no, status,
warranty_until)

stock_movements(id, business_id, branch_id, product_id/variant_id,
                 type [purchase|sale|return|adjustment|transfer],
                 qty_delta, ref_type, ref_id, created_by,
created_at,
                 client_generated_id)   -- idempotency key

customers(id, business_id, name, phone, address, opening_due_minor)
customer_payments(id, customer_id, invoice_id, amount_minor, method,
paid_at)

suppliers(id, business_id, name, phone, address)
purchase_orders(id, business_id, branch_id, supplier_id, status,
total_minor)
purchase_items(id, purchase_order_id, product_id/variant_id, qty,
unit_cost_minor)

sales_invoices(id, business_id, branch_id, invoice_no, customer_id,
                subtotal_minor, discount_minor, tax_minor,
total_minor,
                paid_minor, due_minor, status, created_by,
                client_generated_id, synced_at)
sales_items(id, invoice_id, product_id/variant_id, qty,
```

```
unit_price_minor, discount_minor)
sales_returns(id, invoice_id, items_json, refund_minor, reason,
created_at)

payments(id, business_id, invoice_id/purchase_id, amount_minor,
method, direction, created_at)
expenses(id, business_id, branch_id, category, amount_minor, note,
created_at)
employees(id, business_id, branch_id, name, role, salary_minor)

sync_log(id, business_id, entity_type, entity_id,
client_generated_id,
         status [pending|success|failed|conflict], attempt_count,
         last_error, created_at, resolved_at)
audit_log(id, business_id, user_id, action, entity_type, entity_id,
diff_json, created_at)
```

RLS pattern: every table has `business_id`; policies restrict `SELECT/INSERT/UPDATE` to rows where `business_id = auth.jwt() -> 'business_id'` and, for branch-scoped tables, additionally check branch membership. No hard deletes on `sales_invoices`, `payments`, `stock_movements` — status/void flags only.

---

6. Local Schema (Dexie / IndexedDB)

Mirrors the cloud schema closely so sync mapping is mechanical, plus local-only bookkeeping fields.

```
        // lib/db/dexie/schema.ts
        db.version(1).stores({
          products:        'localId, serverId, sku, barcode, category_id,
business_id, [business_id+is_active]',
          variants:        'localId, serverId, product_localId, sku',
          stockMovements:  'localId, serverId, product_localId, type,
created_at, syncStatus',
          customers:       'localId, serverId, phone, business_id',
          suppliers:       'localId, serverId, business_id',
          salesInvoices:   'localId, serverId, invoice_no, customer_localId,
status, created_at, syncStatus',
          salesItems:      'localId, invoice_localId, product_localId',
          payments:        'localId, serverId, invoice_localId, created_at,
syncStatus',
          expenses:        'localId, serverId, created_at, syncStatus',
          purchaseOrders:  'localId, serverId, supplier_localId, status,
syncStatus',
          syncQueue:       '++id, entityType, entityLocalId, op, status,
attempts, createdAt',
          meta:            'key'   // last sync timestamp, device id,
business/branch context
        });
```

Every record has: - `localId` (client-generated UUID, primary key — never reused) - `serverId` (nullable until first successful sync) - `syncStatus: 'pending' | 'syncing' | 'synced' | 'failed' | 'conflict'` - `updatedAt` for last-write tracking

---

7. Sync Design

Write path: UI action → domain layer validates → single Dexie transaction writes the record(s) + enqueues a `syncQueue` entry, all atomically → UI reflects success immediately.

Sync engine (background, runs on reconnect + interval + manual trigger): 1. Pull oldest `pending` queue entries in dependency order (e.g. customer before invoice referencing it). 2. POST to Supabase using the `client_generated_id` as an idempotency key (upsert semantics — resubmitting a successful op is a no-op). 3. On success: mark `synced`, store returned `serverId`, update local record. 4. On failure (network): mark `failed`, backoff + retry (exponential, capped), stays queued. 5. On conflict (e.g. stock quantity changed server-side since last pull): mark `conflict`, **do not auto-resolve financial records** — surface in a Sync Issues panel for manual review. Inventory conflicts can auto-resolve via movement-log replay (movements are additive deltas, not absolute overwrites, so most stock conflicts self-resolve).

Idempotency: every locally-created entity gets a UUID at creation time; that UUID is the unique constraint on the server table, so retried syncs never duplicate a sale or payment.

Conflict policy (explicit): - Stock: resolved by replaying movement deltas, not by last-write-wins on a quantity field. - Money (sales, payments, refunds): never silently overwritten. Any server-side divergence from a local pending record is flagged for human review before it's applied.

Status UI: a persistent sync indicator (online/offline, N pending, N failed/conflict) with a drill-down list — this is a first-class screen, not a toast.

---

8. Security Model

- Supabase Auth for login; session cached locally so the app opens offline after first login (no auth check blocking local operations).
- Row Level Security on every table, scoped by `business_id`, with branch-level policies layered on top for multi-branch isolation.
- Role-based permissions table (`roles_permissions`) checked both client-side (UX) and server-side (RLS/Edge Functions) — client checks are never the actual security boundary.
- No service-role key ever ships to the browser; privileged operations (e.g. bulk export, user management) go through Supabase Edge Functions.
- `audit_log` captures who changed what, with a diff, for every financial/stock-affecting action.
- Local IndexedDB is per-device/per-business — on logout, local business data is wiped unless the user explicitly opts into "keep offline data on this device."

---

9. Development Phases

Build in this order; each phase ends with working, tested functionality — not a stub.

1. **Foundation** — Next.js + TS strict + Tailwind + shadcn scaffold, Dexie schema, Supabase project + schema + RLS, auth flow (online + cached offline session), service worker + PWA manifest, sync queue skeleton (no business logic yet, just enqueue/dequeue/retry).
2. **Product & Inventory core** — categories, products, variants, units, stock movements, barcode/SKU handling, low-stock logic. Local-first CRUD with sync.
3. **POS + Sales** — cart, checkout, multiple payment methods, partial payments/due, receipt generation (print + PDF), offline invoice numbering strategy (prefix by device/branch to avoid collisions).

4. **Customers & Due Collection** — customer CRUD, due ledger, payment collection, customer statements.
5. **Suppliers & Purchasing** — supplier CRUD, purchase orders, purchase invoices, receiving stock (stock movement generation).
6. **Returns & Exchanges** — sales returns, refund handling, stock reversal movements.
7. **Expenses & Employees** — basic CRUD + reporting hooks.
8. **Dashboard & Reports** — today's/monthly sales, profit, stock valuation, dues, Recharts visualizations, CSV export.
9. **Multi-Branch** — branch-scoped data, cross-branch permission boundaries, branch switcher.
10. **Hardening** — full sync conflict UI, backup/restore/export verification, audit log UI, permission management UI, performance pass (large product catalogs, IndexedDB indexing), full test suite pass.

Each phase: implement → run type checks/tests → fix → only then move on. Phase 1 is the "first task" deliverable.

---

10. Critical Risks

Risk	Mitigation
Duplicate sales from retried syncs	Client-generated UUID as idempotency key on every write
Stock drift between branches/devices	Movement-log (delta) model instead of absolute-quantity overwrites
Silent financial data loss on conflict	Conflicts on money tables always surface for manual resolution, never auto-merged
Invoice number collisions offline	Branch/device-prefixed local numbering, reconciled to a canonical sequence on sync
IndexedDB storage limits / corruption	Periodic verified export, storage quota monitoring, migration versioning in Dexie
Large catalogs slowing local queries	Proper Dexie compound indexes, pagination, avoid full-table scans in POS search
Auth token expiry while offline	Cache session with long-lived refresh handling; never block local writes on token validity

11. Test Strategy

- **Unit:** money math (minor-unit arithmetic), validation schemas, stock movement calculations.
- **Integration:** full sale flow (cart → payment → invoice → stock movement) against a local Dexie instance.
- **Sync tests:** simulate offline writes, reconnect, verify idempotent sync, verify conflict detection on concurrent stock/money changes, verify retry/backoff behavior.
- **E2E (later phase):** POS flow, due collection flow, returns flow, offline → online transition.

---

12. Setup & Environment

```
NEXT_PUBLIC_SUPABASE_URL=
NEXT_PUBLIC_SUPABASE_ANON_KEY=
SUPABASE_SERVICE_ROLE_KEY=        # server-only, never exposed to
client
NEXT_PUBLIC_APP_ENV=development

        npm install
        npm run supabase:migrate   # applies schema + RLS policies
        npm run dev
```

---

13. Instructions for Antigravity (hand this section to it directly)

> Inspect the workspace first. Confirm or adapt the schema and folder structure above to the actual repo state. Implement **Phase 1 only** in this pass: project scaffold, Dexie schema, Supabase schema + RLS, auth (online + offline-cached session), service worker/PWA shell, and the sync queue skeleton (enqueue/dequeue/retry, no business entities synced yet beyond a trivial test entity). Run type checks and any available tests after the phase. Do not implement POS, inventory, or any later-phase module yet. Do not claim production-readiness. Report back what was built, what's stubbed, and what's still missing before continuing to Phase 2.

Prepared for Abrar — StoreOS offline-first store management system.